

Backend Python Source Code

Project: T&C; Red Flag Scanner | Team: CorruptX

File: backend/app.py

Size: 9620 bytes

```

1 | """
2 | T&C Red Flag Scanner - Flask Server
3 | Team: CorruptX
4 | Cybersecurity Hackathon Platform
5 | """
6 |
7 | import os
8 | import sys
9 | import json
10 | from flask import Flask, request, jsonify, send_from_directory, render_template_string
11 | from flask_cors import CORS
12 | from bs4 import BeautifulSoup
13 |
14 | from analyzer import ClauseAnalyzer
15 |
16 | # Setup Flask application
17 | BASE_DIR = os.path.dirname(os.path.abspath(__file__))
18 | FRONTEND_DIR = os.path.abspath(os.path.join(BASE_DIR, "..", "frontend"))
19 |
20 | app = Flask(__name__, static_folder=FRONTEND_DIR, static_url_path="")
21 | CORS(app) # Allow cross-origin requests for local dev / dual-mode hosting
22 |
23 | # Initialize Analyzer engine
24 | analyzer = ClauseAnalyzer(os.path.join(BASE_DIR, "risk_rules.json"))
25 |
26 | # Load Sample Policies
27 | SAMPLE_POLICIES_PATH = os.path.join(BASE_DIR, "sample_policies.json")
28 | try:
29 |     with open(SAMPLE_POLICIES_PATH, "r", encoding="utf-8") as f:
30 |         SAMPLE_POLICIES = json.load(f).get("samples", [])
31 | except Exception as e:
32 |     print(f"Warning: Could not load sample policies: {e}")
33 |     SAMPLE_POLICIES = []
34 |
35 |
36 | # -----
37 | # FRONTEND STATIC ROUTES
38 | # -----
39 |
40 | @app.route("/")
41 | def serve_index():
42 |     return send_from_directory(FRONTEND_DIR, "index.html")
43 |
44 | @app.route("/scanner")
45 | def serve_scanner():
46 |     return send_from_directory(FRONTEND_DIR, "scanner.html")
47 |
48 | @app.route("/results")
49 | def serve_results():
50 |     return send_from_directory(FRONTEND_DIR, "results.html")
51 |
52 | @app.route("/history")
53 | def serve_history():
54 |     return send_from_directory(FRONTEND_DIR, "history.html")
55 |
56 | @app.route("/about")
57 | def serve_about():
58 |     return send_from_directory(FRONTEND_DIR, "about.html")
59 |
60 |

```

```

61 | # -----
62 | # REST API ENDPOINTS
63 | # -----
64 |
65 | @app.route("/api/health", methods=["GET"])
66 | def health_check():
67 |     """Health & capability probe."""
68 |     return jsonify({
69 |         "status": "healthy",
70 |         "app": "T&C Red Flag Scanner",
71 |         "team": "CorruptX",
72 |         "version": "1.0.0",
73 |         "offline_capable": True,
74 |         "engine": "Modular Cybersecurity Rule-Based NLP",
75 |         "active_categories": len(analyzer.categories)
76 |     })
77 |
78 | @app.route("/api/rules", methods=["GET"])
79 | def get_rules():
80 |     """Return active risk taxonomy and scoring rules."""
81 |     return jsonify(analyzer.rules_data)
82 |
83 | @app.route("/api/demo", methods=["GET"])
84 | def get_demo_policies():
85 |     """Return realistic demo policies for instant hackathon evaluation."""
86 |     return jsonify({
87 |         "status": "success",
88 |         "count": len(SAMPLE_POLICIES),
89 |         "samples": SAMPLE_POLICIES
90 |     })
91 |
92 | @app.route("/api/scan", methods=["POST"])
93 | def scan_policy():
94 |     """
95 |     Main analysis endpoint.
96 |     Accepts JSON body: { "text": "...", "document_name": "..." }
97 |     or multipart file upload with key 'file' and optional 'document_name'.
98 |     """
99 |     try:
100 |         raw_text = ""
101 |         doc_name = "Submitted Document"
102 |
103 |         # 1. Check if multipart file upload
104 |         if "file" in request.files:
105 |             uploaded_file = request.files["file"]
106 |
107 |             filename = uploaded_file.filename or "uploaded_policy.txt"
108 |             doc_name = request.form.get("document_name", filename)
109 |
110 |             content_bytes = uploaded_file.read()
111 |             # Try utf-8 decoding, fallback to latin-1
112 |             try:
113 |                 file_content = content_bytes.decode("utf-8")
114 |             except UnicodeDecodeError:
115 |                 file_content = content_bytes.decode("latin-1", errors="ignore")
116 |
117 |             # If HTML, strip tags cleanly with BeautifulSoup
118 |             if filename.lower().endswith((".html", ".htm")):
119 |                 soup = BeautifulSoup(file_content, "html.parser")
120 |                 # Remove scripts and styles
121 |                 for script in soup(["script", "style", "meta", "noscript"]):
122 |                     script.extract()
123 |                 raw_text = soup.get_text(separator="\n\n")
124 |             else:
125 |                 raw_text = file_content
126 |
127 |         # 2. Check if JSON payload
128 |         elif request.is_json:
129 |             data = request.get_json(silent=True) or {}
130 |             raw_text = data.get("text", "")

```

```

130 |         doc_name = data.get("document_name", "Pasted Policy")
131 |
132 |     # 3. Check form data
133 |     elif request.form:
134 |         raw_text = request.form.get("text", "")
135 |         doc_name = request.form.get("document_name", "Submitted Policy")
136 |
137 |     # Edge case: Empty input
138 |     if not raw_text or len(raw_text.strip()) == 0:
139 |         return jsonify({
140 |             "error": "Input text is empty. Please provide Terms & Conditions text or upload a document.",
141 |             "risk_score": 0,
142 |             "risk_level": "SAFE",
143 |             "clauses_analyzed": 0,
144 |             "red_flags": 0
145 |         }), 400
146 |
147 |     # Edge case: Extremely short text
148 |     if len(raw_text.strip()) < 15:
149 |         return jsonify({
150 |             "error": "The supplied text is too short to perform a meaningful Terms & Conditions risk assessment.",
151 |             "risk_score": 0,
152 |             "risk_level": "SAFE",
153 |             "clauses_analyzed": 0,
154 |             "red_flags": 0
155 |         }), 400
156 |
157 |     # Safety limit for hackathon demo (e.g. max 500,000 chars)
158 |     if len(raw_text) > 500000:
159 |         raw_text = raw_text[:500000]
160 |
161 |     # Perform analysis
162 |     result = analyzer.scan(raw_text, doc_name)
163 |     return jsonify(result), 200
164 |
165 | except Exception as e:
166 |     print(f"Error during scan: {str(e)}", file=sys.stderr)
167 |     return jsonify({
168 |         "error": f"Internal scan engine exception: {str(e)}",
169 |         "risk_score": 0,
170 |         "risk_level": "ERROR"
171 |     }), 500
172 |
173 |
174 | @app.route("/api/chat", methods=["POST"])
175 | def chat_assistant():
176 |     """
177 |     AI Privacy Assistant Chatbot Endpoint.
178 |     Accepts JSON body: { "message": "...", "active_scan": {...} }
179 |     """
180 |     try:
181 |         data = request.get_json(silent=True) or {}
182 |         user_msg = data.get("message", "").strip()
183 |         active_scan = data.get("active_scan")
184 |
185 |         if not user_msg:
186 |             return jsonify({"reply": "Please ask a question about Terms & Conditions or digital privacy."}), 400
187 |
188 |         msg_lower = user_msg.lower()
189 |
190 |         # Contextual question about active scan
191 |         if active_scan and any(w in msg_lower for w in ["my document", "my policy", "scan", "scanned", "results", "red f
192 |             doc_name = active_scan.get("document_name", "Uploaded Policy")
193 |             score = active_scan.get("risk_score", 0)
194 |             level = active_scan.get("risk_level", "SAFE")
195 |             flagged = active_scan.get("flagged_clauses", [])
196 |
197 |             if "sell" in msg_lower or "sharing" in msg_lower:
198 |                 data_sell = [c for c in flagged if any(m.get("category_id") in ["DATA_SELLING", "THIRD_PARTY_SHARING"] f

```

```

199 |         if data_sell:
200 |             snippet = data_sell[0].get("text", "")[:150]
201 |             reply = f"■ <b>Data Selling Warning for '{doc_name}':</b><br><br>Clause ma
202 |         else:
203 |             reply = f"■ <b>No explicit Data Selling clauses</b> detected in <b>'{doc_name}'</b>
204 |             return jsonify({"reply": reply})
205 |
206 |         reply = f"■ <b>Analysis Summary for '{doc_name}':</b>• <b>Risk Score:</b>
207 |         return jsonify({"reply": reply})
208 |
209 |     # General Knowledge Answers
210 |     if "arbitration" in msg_lower or "sue" in msg_lower or "court" in msg_lower:
211 |         reply = "■ <b>Mandatory Arbitration Clauses</b> strip away your constitutional right to take a
212 |     elif "sell" in msg_lower or "monetiz" in msg_lower or "third party" in msg_lower:
213 |         reply = "■ <b>Data Selling Clauses</b> allow companies to package and monetize your personal det
214 |     elif "gdpr" in msg_lower or "ccpa" in msg_lower or "right" in msg_lower:
215 |         reply = "■ <b>Digital Privacy Laws (GDPR/CCPA):</b> You have the legal right to access your da
216 |     else:
217 |         reply = f"I reviewed your question regarding <b>'{user_msg}'</b>.<br><br>Always look
218 |
219 |     return jsonify({"reply": reply}), 200
220 |
221 | except Exception as e:
222 |     return jsonify({"reply": f"Sorry, I encountered an error processing your query: {str(e)}"}), 500
223 |
224 |
225 | # -----
226 | # ENTRY POINT
227 | # -----
228 |
229 | if __name__ == "__main__":
230 |     port = int(os.environ.get("PORT", 5000))
231 |     print("=" * 60)
232 |     print("  CORRUPTX - T&C RED FLAG SCANNER")
233 |     print("  Cybersecurity Hackathon Engine")
234 |     print(f"  Serving on http://127.0.0.1:{port}")
235 |     print("  Offline capable: Yes (zero external cloud dependencies)")
236 |     print("=" * 60)
237 |     app.run(host="0.0.0.0", port=port, debug=True)

```

File: backend/analyzer.py

Size: 17659 bytes

```

1 | """
2 | T&C Red Flag Scanner - Analysis Engine
3 | Team: CorruptX
4 | Modular rule-based NLP and risk taxonomy engine for Terms & Conditions, Privacy Policies, and EULAs.
5 | Designed for 100% offline capability and drop-in extensibility for future LLM pipelines.
6 | """
7 |
8 | import re
9 | import json
10 | import os
11 | from datetime import datetime
12 | from typing import Dict, List, Any, Optional
13 |
14 | class ClauseAnalyzer:
15 |     """
16 |     Modular analysis engine that segments legal text into clauses,
17 |     matches them against a cybersecurity risk taxonomy, and computes
18 |     a deterministic 0-100 risk score with plain-English security explanations.
19 |     """
20 |
21 |     def __init__(self, rules_path: Optional[str] = None):
22 |         if rules_path is None:
23 |             # Default to adjacent risk_rules.json
24 |             current_dir = os.path.dirname(os.path.abspath(__file__))
25 |             rules_path = os.path.join(current_dir, "risk_rules.json")
26 |
27 |         self.rules_path = rules_path
28 |         self.rules_data = self._load_rules()
29 |         self.categories = self.rules_data.get("categories", [])
30 |         self._compile_patterns()
31 |
32 |     def _load_rules(self) -> Dict[str, Any]:
33 |         """Load the cybersecurity taxonomy from JSON."""
34 |         if not os.path.exists(self.rules_path):
35 |             raise FileNotFoundError(f"Risk rules taxonomy file not found: {self.rules_path}")
36 |
37 |         with open(self.rules_path, "r", encoding="utf-8") as f:
38 |             return json.load(f)
39 |
40 |     def _compile_patterns(self):
41 |         """Pre-compile regex patterns for high-throughput clause evaluation."""
42 |         self.compiled_rules = []
43 |         for cat in self.categories:
44 |             compiled_patterns = []
45 |             for pattern_str in cat.get("patterns", []):
46 |                 try:
47 |                     compiled_patterns.append(re.compile(pattern_str, re.IGNORECASE))
48 |                 except re.error as e:
49 |                     print(f"Warning: Invalid regex in category {cat.get('id')}: {pattern_str} ({e})")
50 |
51 |             self.compiled_rules.append({
52 |                 "id": cat.get("id"),
53 |                 "name": cat.get("name"),
54 |                 "severity": cat.get("severity", "LOW"),
55 |                 "weight": cat.get("weight", 5),
56 |                 "color": cat.get("color", "#38bdf8"),
57 |                 "explanation": cat.get("explanation", ""),
58 |                 "why_it_matters": cat.get("why_it_matters", ""),
59 |                 "recommendation": cat.get("recommendation", ""),
60 |                 "keywords": [k.lower() for k in cat.get("keywords", [])],
61 |                 "patterns": compiled_patterns
62 |             })
63 |
64 |     def clean_text(self, raw_text: str) -> str:
65 |         """Sanitize and normalize text formatting without losing legal sentence boundaries."""
66 |         if not raw_text:
67 |             return ""

```

```

68 |         # Replace smart quotes and dashes
69 |         cleaned = raw_text.replace("'", "\'").replace('"', "\"")
70 |         cleaned = cleaned.replace("'", "'").replace('"', "\"")
71 |         cleaned = cleaned.replace("-", " - ").replace("- ", " - ")
72 |
73 |         # Normalize carriage returns and excessive tabs
74 |         cleaned = cleaned.replace("\r\n", "\n").replace("\r", "\n")
75 |
76 |         # Remove null bytes or non-printable control characters
77 |         cleaned = re.sub(r"[\x00-\x08\x0b\x0c\x0e-\x1f]", "", cleaned)
78 |
79 |         return cleaned.strip()
80 |
81 | def segment_clauses(self, text: str) -> List[str]:
82 |     """
83 |     Segment policy text into distinct clauses or sentences while preserving
84 |     numbered sections (e.g. '1.1', 'Section 4(a)') and common abbreviations.
85 |     """
86 |     cleaned = self.clean_text(text)
87 |     if not cleaned:
88 |         return []
89 |
90 |     # Mask known abbreviations that contain periods
91 |     abbr_map = {
92 |         r"\be\.g\.\.b": "EG_TEMP_MARKER",
93 |         r"\bi\.e\.\.b": "IE_TEMP_MARKER",
94 |         r"\bU\.S\.\.b": "US_TEMP_MARKER",
95 |         r"\betc\.\.b": "ETC_TEMP_MARKER",
96 |         r"\bInc\.\.b": "INC_TEMP_MARKER",
97 |         r"\bLtd\.\.b": "LTD_TEMP_MARKER",
98 |         r"\bLLC\.\.b": "LLC_TEMP_MARKER",
99 |         r"\bSec\.\.b": "SEC_TEMP_MARKER",
100 |         r"\bNo\.\.b": "NO_TEMP_MARKER",
101 |         r"\bv\.\.b": "V_TEMP_MARKER",
102 |         r"\bvs\.\.b": "VS_TEMP_MARKER"
103 |     }
104 |
105 |     masked = cleaned
106 |     for regex_pat, replacement in abbr_map.items():
107 |         masked = re.sub(regex_pat, replacement, masked, flags=re.IGNORECASE)
108 |
109 |     # Protect section numbering like 1.1 or 3.2.1
110 |     masked = re.sub(r"(<= \d) \. (?= \d)", "DECIMAL_DOT_MARKER", masked)
111 |
112 |     # Split on double newlines (paragraphs/clauses) or sentence-ending punctuation followed by space/newline
113 |     # or numbered list items like \n1. or \n(a)
114 |     chunks = re.split(r"(?:\n\s*\n|(<= [! ?])\s+(?=[A-Z0-9\"'])|(<= \n)\s*(?=[0-9]+\.\s|[A-Z]\.\s|([a-z0-9]+)+\s|$))", masked)
115 |
116 |     clauses = []
117 |     for chunk in chunks:
118 |         # Restore markers
119 |         restored = chunk
120 |         for regex_pat, replacement in abbr_map.items():
121 |             marker_target = replacement
122 |             original_sample = regex_pat.replace(r"\b", "").replace("\\", "\\")
123 |             restored = restored.replace(marker_target, original_sample)
124 |
125 |         restored = restored.replace("DECIMAL_DOT_MARKER", ".")
126 |         restored = re.sub(r"\s+", " ", restored).strip()
127 |
128 |         # Avoid single stray numbers or trivial fragments
129 |         if len(restored) >= 12 or re.search(r"[a-zA-Z]{3,}", restored):
130 |             clauses.append(restored)
131 |
132 |     # If splitting resulted in no valid clauses but text was present, use full text
133 |     if not clauses and len(cleaned) > 0:
134 |         clauses = [cleaned]
135 |
136 |     return clauses

```

```

137 |
138 | def is_negated_protection(self, clause_text: str, category_id: str) -> bool:
139 |     """
140 |     Detect if the clause explicitly promises protection via negation
141 |     (e.g., 'we do not sell your data', 'we do not use device fingerprinting', 'never track').
142 |     Protective clauses should not be flagged as red flags.
143 |     """
144 |     clause_lower = clause_text.lower()
145 |
146 |     # Specific negative assertions by category that represent protections rather than risks
147 |     negation_rules = {
148 |         "data_sharing": [
149 |             r"(?:we|company|our\s+service)\s+(?:do\s+not|will\s+not|does\s+not|never)\s+(?:sell|share|disclose|rent|
150 |             r"no\s+(?:third-party\s+sale|selling|sharing\s+with\s+third\s+parties)",
151 |             r"(?:we\s+do\s+not|never)\s+disclose\s+personal\s+data"
152 |         ],
153 |         "tracking": [
154 |             r"(?:we|company|our\s+service)\s+(?:do\s+not|will\s+not|does\s+not|never)\s+(?:track|use\s+tracking|recor
155 |             r"(?:no|zero)\s+(?:tracking|fingerprinting|telemetry|session\s+replay)",
156 |             r"do\s+not\s+use\s+(?:tracking\s+cookies|device\s+fingerprinting|analytics\s+pixels)"
157 |         ],
158 |         "data_collection": [
159 |             r"(?:we|company)\s+(?:do\s+not|will\s+not|does\s+not|never)\s+collect\s+(?:biometric|location|sensitive|
160 |             r"do\s+not\s+collect\s+location\s+data"
161 |         ],
162 |         "location_tracking": [
163 |             r"(?:we|company)\s+(?:do\s+not|will\s+not|does\s+not|never)\s+(?:collect|track)\s+(?:your\s+)?location",
164 |             r"do\s+not\s+collect\s+location\s+data"
165 |         ],
166 |         "advertising_profiling": [
167 |             r"no\s+advertising\s+networks\s+are\s+embedded",
168 |             r"(?:we|company)\s+(?:do\s+not|will\s+not|never)\s+(?:serve\s+targeted|profile|sell\s+ads)"
169 |         ],
170 |         "arbitration": [
171 |             r"(?:we|company)\s+(?:do\s+not|will\s+not)\s+enforce\s+(?:mandatory\s+)?(?:binding\s+)?arbitration",
172 |             r"do\s+not\s+enforce\s+mandatory\s+binding\s+arbitration"
173 |         ]
174 |     }
175 |
176 |     cat_negations = negation_rules.get(category_id, [])
177 |     for pat in cat_negations:
178 |         if re.search(pat, clause_lower):
179 |             return True
180 |     return False
181 |
182 | def analyze_clause(self, clause_text: str, index: int) -> Dict[str, Any]:
183 |     """
184 |     Evaluate a single clause against the cybersecurity rule taxonomy.
185 |     Returns matched risk details or clean status.
186 |     """
187 |     clause_lower = clause_text.lower()
188 |     matched_categories = []
189 |
190 |     for rule in self.compiled_rules:
191 |         # First check if this clause is explicitly a privacy protection for this category
192 |         if self.is_negated_protection(clause_text, rule["id"]):
193 |             continue
194 |
195 |         matched_flag = False
196 |         matched_pattern_str = None
197 |
198 |         # 1. Test regex patterns
199 |         for pattern in rule["patterns"]:
200 |             match = pattern.search(clause_text)
201 |             if match:
202 |                 matched_flag = True
203 |                 matched_pattern_str = match.group(0)
204 |                 break
205 |

```

```

206 |         # 2. Test keywords if regex didn't trigger
207 |         if not matched_flag:
208 |             for kw in rule["keywords"]:
209 |                 if kw in clause_lower:
210 |                     matched_flag = True
211 |
212 |                     matched_pattern_str = kw
213 |                     break
214 |
215 |         if matched_flag:
216 |             matched_categories.append({
217 |                 "category_id": rule["id"],
218 |                 "category_name": rule["name"],
219 |                 "severity": rule["severity"],
220 |                 "weight": rule["weight"],
221 |                 "color": rule["color"],
222 |                 "explanation": rule["explanation"],
223 |                 "why_it_matters": rule["why_it_matters"],
224 |                 "recommendation": rule["recommendation"],
225 |                 "snippet": matched_pattern_str
226 |             })
227 |
228 |     severity_rank = {"HIGH": 3, "MEDIUM": 2, "LOW": 1}
229 |
230 |     if matched_categories:
231 |         # Sort by severity weight descending
232 |         matched_categories.sort(
233 |             key=lambda x: (severity_rank.get(x["severity"], 0), x["weight"]),
234 |             reverse=True
235 |         )
236 |         top_match = matched_categories[0]
237 |
238 |         return {
239 |             "index": index,
240 |             "text": clause_text,
241 |             "is_flagged": True,
242 |             "category_id": top_match["category_id"],
243 |             "category_name": top_match["category_name"],
244 |             "severity": top_match["severity"],
245 |             "weight": top_match["weight"],
246 |             "color": top_match["color"],
247 |             "explanation": top_match["explanation"],
248 |             "why_it_matters": top_match["why_it_matters"],
249 |             "recommendation": top_match["recommendation"],
250 |             "all_matched_categories": [m["category_name"] for m in matched_categories]
251 |         }
252 |     else:
253 |         return {
254 |             "index": index,
255 |             "text": clause_text,
256 |             "is_flagged": False,
257 |             "category_id": None,
258 |             "category_name": "Standard Clause",
259 |             "severity": "SAFE",
260 |             "weight": 0,
261 |             "color": "#00ff88",
262 |             "explanation": "No elevated risk indicators detected in this clause.",
263 |             "why_it_matters": "Standard service provision or routine operational statement.",
264 |             "recommendation": "Clause appears customary; verify context if unsure.",
265 |             "all_matched_categories": []
266 |         }
267 |
268 | def calculate_risk_score(self, total_clauses: int, flagged_clauses: List[Dict[str, Any]]) -> int:
269 |     """
270 |     Compute a reproducible 0-100 risk score based on:
271 |     - Quantity and severity weights of detected red flags
272 |     - Risk density relative to total document clauses
273 |     - Guardrail thresholds (e.g. presence of multiple HIGH risks)
274 |     """
275 |     if total_clauses <= 0 or not flagged_clauses:

```



```

275 |         return 5 # Baseline minimal score for empty/neutral policies
276 |
277 |     high_count = sum(1 for c in flagged_clauses if c["severity"] == "HIGH")
278 |     medium_count = sum(1 for c in flagged_clauses if c["severity"] == "MEDIUM")
279 |     low_count = sum(1 for c in flagged_clauses if c["severity"] == "LOW")
280 |
281 |     # Base weighted sum
282 |     weighted_sum = (high_count * 16) + (medium_count * 8) + (low_count * 4)
283 |
284 |     # Density factor: proportion of document containing red flags (up to 20 pts)
285 |     density = len(flagged_clauses) / max(total_clauses, 1)
286 |     density_bonus = min(20, int(density * 35))
287 |
288 |     # Composite score
289 |     raw_score = int((weighted_sum * 0.82) + density_bonus)
290 |
291 |     # Guardrails for high-severity issues
292 |     if high_count >= 3:
293 |         raw_score = max(raw_score, 68)
294 |     elif high_count >= 1:
295 |         raw_score = max(raw_score, 42)
296 |     elif medium_count >= 2:
297 |         raw_score = max(raw_score, 35)
298 |
299 |     # Final bounding to 0-100
300 |     score = max(0, min(100, raw_score))
301 |     return score
302 |
303 | def determine_risk_level(self, score: int) -> str:
304 |     """Map 0-100 numerical score to formal risk tier."""
305 |     if score <= 30:
306 |         return "SAFE"
307 |     elif score <= 60:
308 |         return "CAUTION"
309 |     else:
310 |         return "HIGH RISK"
311 |
312 | def scan(self, text: str, document_name: str = "Submitted Policy") -> Dict[str, Any]:
313 |     """
314 |     Execute full end-to-end policy scan.
315 |     Returns complete structured analysis ready for JSON transmission.
316 |
317 |     """
318 |     if not text or len(text.strip()) == 0:
319 |         return {
320 |             "error": "Input text is empty. Please paste or upload a policy to scan.",
321 |             "risk_score": 0,
322 |             "risk_level": "SAFE",
323 |             "clauses_analyzed": 0,
324 |             "red_flags": 0
325 |         }
326 |
327 |     clauses = self.segment_clauses(text)
328 |     total_clauses = len(clauses)
329 |
330 |     analyzed_clauses = []
331 |     flagged_clauses = []
332 |
333 |     category_counts: Dict[str, Dict[str, Any]] = {}
334 |
335 |     for idx, clause in enumerate(clauses):
336 |         res = self.analyze_clause(clause, idx)
337 |         analyzed_clauses.append(res)
338 |         if res["is_flagged"]:
339 |             flagged_clauses.append(res)
340 |             cat_id = res["category_id"]
341 |             if cat_id not in category_counts:
342 |                 category_counts[cat_id] = {
343 |                     "category_id": cat_id,

```

```

344 |         "severity": res["severity"],
345 |         "color": res["color"],
346 |         "count": 0,
347 |         "clauses": []
348 |     }
349 |     category_counts[cat_id]["count"] += 1
350 |     category_counts[cat_id]["clauses"].append(res["index"])
351 |
352 |     score = self.calculate_risk_score(total_clauses, flagged_clauses)
353 |     level = self.determine_risk_level(score)
354 |
355 |     severity_counts = {
356 |         "HIGH": sum(1 for c in flagged_clauses if c["severity"] == "HIGH"),
357 |         "MEDIUM": sum(1 for c in flagged_clauses if c["severity"] == "MEDIUM"),
358 |         "LOW": sum(1 for c in flagged_clauses if c["severity"] == "LOW")
359 |     }
360 |
361 |     # Format sorted categories for breakdown charts
362 |     sorted_categories = sorted(
363 |         list(category_counts.values()),
364 |         key=lambda x: (x["count"], 3 if x["severity"] == "HIGH" else (2 if x["severity"] == "MEDIUM" else 1)),
365 |         reverse=True
366 |     )
367 |
368 |     summary_msg = self._generate_summary(score, level, severity_counts, total_clauses)
369 |
370 |     return {
371 |         "document_name": document_name,
372 |         "analyzed_at": datetime.now().isoformat(),
373 |         "risk_score": score,
374 |         "risk_level": level,
375 |         "summary": summary_msg,
376 |         "clauses_analyzed": total_clauses,
377 |         "red_flags": len(flagged_clauses),
378 |         "severity_counts": severity_counts,
379 |         "categories": sorted_categories,
380 |         "flagged_clauses": flagged_clauses,
381 |         "all_clauses": analyzed_clauses,
382 |         "disclaimer": "This tool provides automated cybersecurity risk analysis for informational purposes and is not intended for legal advice."
383 |     }
384 |
385 | def _generate_summary(self, score: int, level: str, severities: Dict[str, int], total_clauses: int) -> str:
386 |     """Construct a high-level executive security summary."""
387 |     high = severities.get("HIGH", 0)
388 |     med = severities.get("MEDIUM", 0)
389 |     low = severities.get("LOW", 0)
390 |     total_flags = high + med + low
391 |
392 |     if level == "HIGH RISK":
393 |         return (f"CRITICAL PRIVACY & LEGAL EXPOSURE DETECTED. Analysis identified {total_flags} red flags "
394 |                 f"({high} High-Risk clauses) across {total_clauses} clauses. This policy contains terms that may "
395 |                 f"severely compromise personal data privacy, waive legal remedies, or grant unilateral vendor control.")
396 |     elif level == "CAUTION":
397 |         return (f"MODERATE USER EXPOSURE DETECTED. Identified {total_flags} flagged clauses ({high} High, {med} Medium, "
398 |                 f"{low} Low) out of {total_clauses} total clauses. Contains clauses such as recurring billing, data logging, "
399 |                 f"or liability disclaimers that warrant conscious review before agreeing.")
400 |     else:
401 |         return (f"ACCEPTABLE RISK PROFILE. Minimal legal or data exposure detected across {total_clauses} clauses. "
402 |                 f"Only {total_flags} minor items noted. Policy adheres to standard or user-friendly privacy practices.")

```

File: backend/test_server.py

Size: 4935 bytes

```

1 | """
2 | Automated Test Suite for T&C Red Flag Scanner
3 | Tests Flask API endpoints, static serving, input validation, and scoring consistency.
4 | """
5 |
6 | import unittest
7 | import json
8 | import os
9 | from app import app, analyzer
10 |
11 | class TestCRedFlagScanner(unittest.TestCase):
12 |     @classmethod
13 |     def setUpClass(cls):
14 |         cls.client = app.test_client()
15 |
16 |     def test_health_check(self):
17 |         """Verify /api/health returns healthy and correct taxonomy size."""
18 |         response = self.client.get("/api/health")
19 |         self.assertEqual(response.status_code, 200)
20 |         data = response.get_json()
21 |         self.assertEqual(data.get("status"), "healthy")
22 |         self.assertEqual(data.get("team"), "CorruptX")
23 |         self.assertEqual(data.get("offline_capable"), True)
24 |         self.assertEqual(data.get("active_categories"), 14)
25 |
26 |     def test_rules_endpoint(self):
27 |         """Verify /api/rules returns all 14 categories."""
28 |         response = self.client.get("/api/rules")
29 |         self.assertEqual(response.status_code, 200)
30 |         data = response.get_json()
31 |         self.assertIn("categories", data)
32 |         self.assertEqual(len(data["categories"]), 14)
33 |
34 |     def test_demo_endpoint(self):
35 |         """Verify /api/demo returns preloaded policies."""
36 |
37 |         response = self.client.get("/api/demo")
38 |         self.assertEqual(response.status_code, 200)
39 |         data = response.get_json()
40 |         self.assertEqual(data.get("status"), "success")
41 |         self.assertGreaterEqual(len(data.get("samples", [])), 3)
42 |
43 |     def test_scan_empty_input(self):
44 |         """Verify /api/scan rejects empty input with friendly error."""
45 |         response = self.client.post("/api/scan", json={"text": "", "document_name": "Empty"})
46 |         self.assertEqual(response.status_code, 400)
47 |         data = response.get_json()
48 |         self.assertIn("error", data)
49 |
50 |     def test_scan_too_short_input(self):
51 |         """Verify /api/scan rejects input that is too short."""
52 |         response = self.client.post("/api/scan", json={"text": "Hello world", "document_name": "Short"})
53 |         self.assertEqual(response.status_code, 400)
54 |         data = response.get_json()
55 |         self.assertIn("error", data)
56 |
57 |     def test_scan_high_risk_policy(self):
58 |         """Verify scan on high-risk clauses detects categories and assigns HIGH RISK."""
59 |         high_risk_text = (
60 |             "We reserve the right to share your personal information with third parties. "
61 |             "We collect information about your activity across third-party websites to build demographic profiles. "
62 |             "We track your physical location even when the app is closed. "
63 |             "Subscriptions automatically renew unless cancelled. "
64 |             "All disputes shall be resolved exclusively through confidential binding arbitration. "
65 |             "You waive any right to participate in a class action. "
66 |             "We reserve the right to modify these terms at any time without prior notice."
67 |         )
68 |         response = self.client.post("/api/scan", json={

```

```

68 |         "text": high_risk_text,
69 |         "document_name": "Test High Risk"
70 |     })
71 |     self.assertEqual(response.status_code, 200)
72 |     data = response.get_json()
73 |     self.assertEqual(data.get("risk_level"), "HIGH RISK")
74 |     self.assertGreaterEqual(data.get("risk_score"), 68)
75 |     self.assertGreaterEqual(data.get("red_flags"), 5)
76 |     self.assertGreaterEqual(data.get("severity_counts", {}).get("HIGH", 0), 3)
77 |
78 |     def test_scan_safe_policy_with_negation(self):
79 |         """Verify scan on privacy-protective clauses with negation returns SAFE."""
80 |         safe_text = (
81 |             "AegisGuard operates as a local-first software utility. "
82 |             "We do not use tracking cookies, device fingerprinting, analytics pixels, or session replay scripts. "
83 |             "We do not collect location data, contact lists, or background sensor information. "
84 |             "We do not sell, rent, monetize, or disclose personal data to advertisers. "
85 |             "You retain full ownership and intellectual property rights in all data. "
86 |             "We do not enforce mandatory binding arbitration or prohibit collective remedies."
87 |         )
88 |         response = self.client.post("/api/scan", json={
89 |             "text": safe_text,
90 |             "document_name": "Test Safe"
91 |         })
92 |         self.assertEqual(response.status_code, 200)
93 |         data = response.get_json()
94 |         self.assertEqual(data.get("risk_level"), "SAFE")
95 |         self.assertLessEqual(data.get("risk_score"), 30)
96 |
97 |     def test_static_routes(self):
98 |         """Verify static pages are served correctly."""
99 |         routes = ["/", "/scanner", "/results", "/history", "/about"]
100 |         for route in routes:
101 |             response = self.client.get(route)
102 |             self.assertEqual(response.status_code, 200, f"Failed on route: {route}")
103 |             self.assertIn("CorruptX", response.get_data(as_text=True))
104 |
105 | if __name__ == "__main__":
106 |     unittest.main()

```

File: backend/test_analyzer.py**Size:** 768 bytes

```
1 | from analyzer import ClauseAnalyzer
2 | import json
3 |
4 | analyzer = ClauseAnalyzer()
5 | with open('sample_policies.json', 'r', encoding='utf-8') as f:
6 |     samples = json.load(f)['samples']
7 |
8 | print("--- TESTING CLAUSE ANALYZER ---")
9 | for sample in samples:
10 |     res = analyzer.scan(sample['content'], sample['title'])
11 |     print(f"[{res['risk_level']}] {sample['title']}")
12 |     print(f"  Score: {res['risk_score']}/100")
13 |     print(f"  Clauses: {res['clauses_analyzed']}, Red Flags: {res['red_flags']}")
14 |     print(f"  Severity: High={res['severity_counts']['HIGH']}, Med={res['severity_counts']['MEDIUM']}, Low={res['severity_counts']['LOW']}")
15 |     print(f"  Top categories: {[c['category_name'] for c in res['categories'][:3]]}")
16 |     print()
17 |
18 | print("ALL SAMPLES TESTED SUCCESSFULLY!")
```

File: backend/test_client_js.py

Size: 1401 bytes

```
1 | import js2py
2 |
3 | # Polyfill minimal browser environment
4 | context = js2py.EvalJs()
5 | context.execute("""
6 | var window = {
7 |   TC_RISK_RULES: null,
8 |   clientClauseAnalyzer: null
9 | };
10 | var console = {
11 |   warn: function() {},
12 |   log: function() {},
13 |   error: function() {}
14 | };
15 | """)
16 |
17 | with open('../frontend/js/rules_data.js', 'r', encoding='utf-8') as f:
18 |     context.execute(f.read())
19 |
20 | with open('../frontend/js/client_analyzer.js', 'r', encoding='utf-8') as f:
21 |     context.execute(f.read())
22 |
23 | sample_text = (
24 |     "We reserve the right to share your personal information with third parties. "
25 |     "We track your browsing activity and device fingerprinting across external websites. "
26 |     "All disputes shall be resolved exclusively through confidential binding arbitration. "
27 |     "You waive any right to participate in a class action. "
28 |     "We may modify these terms at any time without prior notice."
29 | )
30 |
31 | context.set('sample_text', sample_text)
32 | res = context.eval('window.clientClauseAnalyzer.scan(sample_text, "JS Offline Test")')
33 |
34 | print("--- CLIENT-SIDE JS ENGINE TEST RESULT ---")
35 | print("Document:", res['document_name'])
36 | print("Risk Score:", res['risk_score'], "/ 100")
37 | print("Risk Level:", res['risk_level'])
38 | print("Clauses Analyzed:", res['clauses_analyzed'])
39 | print("Red Flags Detected:", res['red_flags'])
40 | print("Engine Mode:", res['engine_mode'])
41 | print("SUCCESS: 100% OFFLINE CLIENT-SIDE JS ENGINE IS OPERATIONAL!")
```

File: backend/test_chatbot_and_storage.py

Size: 1546 bytes

```

1 | import unittest
2 | import json
3 | import os
4 | import sys
5 |
6 | BASE_DIR = os.path.dirname(os.path.abspath(__file__))
7 | sys.path.insert(0, BASE_DIR)
8 |
9 | from app import app
10 |
11 | class TestChatbotAndStorage(unittest.TestCase):
12 |     def setUp(self):
13 |         self.client = app.test_client()
14 |         self.client.testing = True
15 |
16 |     def test_chat_general_query(self):
17 |         response = self.client.post("/api/chat", json={
18 |             "message": "What is forced arbitration?"
19 |         })
20 |         self.assertEqual(response.status_code, 200)
21 |         data = json.loads(response.data)
22 |         self.assertIn("reply", data)
23 |         self.assertIn("Arbitration", data["reply"])
24 |
25 |     def test_chat_scanned_doc_context(self):
26 |         active_scan = {
27 |             "document_name": "Test Social App Policy",
28 |             "risk_score": 85,
29 |             "risk_level": "HIGH RISK",
30 |             "flagged_clauses": [
31 |                 {
32 |                     "text": "We may sell and monetize user data to third party advertisers.",
33 |                     "matches": [{"category_id": "DATA_SELLING", "category_name": "Data Selling"}]
34 |                 }
35 |             ]
36 |         }
37 |
38 |         response = self.client.post("/api/chat", json={
39 |             "message": "Does my scanned document sell data?",
40 |             "active_scan": active_scan
41 |         })
42 |         self.assertEqual(response.status_code, 200)
43 |         data = json.loads(response.data)
44 |         self.assertIn("Data Selling", data["reply"])
45 |         self.assertIn("Test Social App Policy", data["reply"])
46 |
47 | if __name__ == "__main__":
48 |     unittest.main()

```